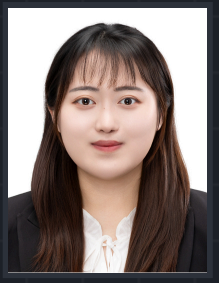
RESUME · 이력서

하지혜 Ha Ji-hye

고객·구독·권한 데이터를 설계하고 운영해 온 3년차 Java 백엔드 개발자

B2B SaaS 백엔드 단독 설계·운영 2년 11개월 · 어드민 화면까지 직접 개발



01 인적사항

성명	하지혜 (Ha Ji-hye)	**지원 분야**	CRM 웹 개발 (Java)
연락처	010-2882-5447	**이메일**	hajh5447@naver.com
기술 블로그	dev-annieha.tistory.com	**GitHub**	github.com/AnnieHa1002
총 경력	2년 11개월 (백엔드)	**최종 학력**	한국외국어대학교 졸업 (학사)

02 학력사항

기간	학교명	전공	구분
2016.03 – 2022.08	한국외국어대학교	언어인지과학과 (부전공: 통계학)	졸업

03 경력사항

기간	회사명	부서 · 직무	담당 업무
2022.11 – 2025.09 2년 11개월	주식회사 콘파파 B2B SaaS 스타트업	IT개발팀 Backend Engineer	• REAT 백엔드 단독 설계·운영 • 어드민·운영 데이터 시스템 풀스택 개발 • TOONDA 데스크톱 앱(C++/Qt) 개발

04 교육사항

기간	교육 과정	기관	비고
2026.02 – 2026.05	AI 활용 백엔드 아키텍처 심화 (KDT)	내일배움캠프	팀장 · 최종 우수상
2022.07 – 2022.10	항해99 8기 Java Spring 과정	팀스파르타	감사상 · 소통왕상

05 자격 · 어학

취득일	구분	명칭	등급
2025.12	어학	OPIc 영어	IH (Intermediate High)

06 핵심 역량

고객 도메인 설계	멀티테넌시·다층 권한, 구독 결제·다통화, 라이선스 데이터 모델링
운영 도구 개발	React 어드민과 Looker Studio 지표 대시보드로 운영팀 지원
정합성 · 성능	보상 트랜잭션·캐시 무효화·분산 락, 쿼리 75 → 5개, 검색 77% 개선

07 기술 스택

Backend	Java 17, Spring Boot 3, Spring Security, Spring Data JPA, QueryDSL, jOOQ, JUnit5
Frontend	CSS, JavaScript, React(어드민 웹), SSE(EventSource), C++/Qt(데스크톱 UI)
Database	PostgreSQL, MySQL, Redis, DynamoDB
외부 연동	PortOne(PG 구독 결제), OAuth2(Google·Kakao·Apple), FCM, AWS SES·SMTP, OpenAI API
Infra · DevOps	AWS(EC2, RDS, S3, CloudFront, Lambda, Cognito), GCP(Compute Engine, Cloud SQL), Docker, Jenkins, GitHub Actions
메시징 · 도구	Kafka, Prometheus, Grafana, Loki, Zipkin, K6, Gatling, Looker Studio, Git, Postman, Claude, Cursor

08 경력 요약 — 주식회사 콘파파 주요 성과

고객 계정 · 권한	워크스페이스 멀티테넌시와 다층 권한 설계, 권한 판단 로직 중앙화로 오류 재발 차단
구독 · 결제	PortOne 빌링키 구독 결제 — 신규·갱신·플랜/좌석 변경·재청구·쿠폰·다통화 통합, 운영 중 오류 1건(네트워크 제외)
라이선스 · 프로모션	엔터프라이즈 고객 라이선스, 쿠폰·레퍼럴 코드, AI 사용량 집계·과금 분석
어드민 · 데이터	React 어드민(라이선스 관리·워크스페이스 이전), Looker Studio 서비스 지표 대시보드
고객 알림	Polling → SSE 실시간 알림 전환, Redis 기반 유실 재전송, 수신 설정 7종 비트마스크 통합
조회 성능	JOIN FETCH + Redis Cache-Aside로 리스트 쿼리 75 → 5개 (93% 감소)
대용량 처리	이미지 처리 Lambda 분리·병렬화로 20장 기준 5분 → 15초, 고해상도 업로드 OOM 해소
아키텍처 · 인프라	Lambda 74개 → Spring Boot 중앙 서버 전환(PoC 3일), AWS → GCP 이전으로 월 150만 → 3만원(98% 절감), Jenkins CI/CD

09 주요 프로젝트

기간	프로젝트	역할	핵심 내용
2026.09 – 진행 중	나공주 키우기 캐릭터 육성형 집중 앱	백엔드 설계 (2인)	오프라인 퍼스트 동기화, 멱등키 기반 중복 지급 방지 설계 (2027.03 출시 예정)
2026.04 – 2026.05	고글에듀 자격증·멘토링 예약 플랫폼	6인 팀 리더	Redisson 분산 락, GIN 트라이그램 검색 77% 개선, 최종 프로젝트 우수상
2026.03 – 2026.04	팔로우미 배송 MSA 물류 서비스 (2주)	6인 팀 리더	Kafka Outbox/Inbox 공통 라이브러리, JPA 쓰기 · jOOQ 조회 분리
2025.05 – 2025.11	Trobl 부부싸움 녹음·분석 앱	백엔드 · 어드민 웹	React 어드민, FCM 즉시·배치·예약 알림, 소셜 로그인 3종 통합 · App Store 출시

CAREER DESCRIPTION

경력기술서

PROJECT 01 · 주식회사 콘파파

REAT — 이미지 기반 피드백 B2B SaaS

2024.04 - 2025.09

서비스	웹툰 작가와 편집자가 긴 이미지 위에 피드백을 남기고 버전별로 관리하는 협업 서비스. 웹툰 제작사와 일본 고객을 대상으로 운영했습니다.
역할	MVP 이후 정식 서비스의 백엔드를 단독으로 설계·운영, 3개월 만에 출시
기술	Java, Spring Boot, PostgreSQL, Redis, Jenkins, Docker, AWS(EC2·RDS·S3·CloudFront·Lambda·Cognito), GCP(Compute Engine·Cloud SQL)

01 워크스페이스 멀티테넌시 · 다층 권한 설계

문제	출시 한 달 전 비즈니스 모델이 바뀌면서 권한 체계를 급히 고쳐야 했습니다. 권한 판단이 코드 여러 곳에 흩어져 있어 수정 후 크고 작은 권한 오류가 이어졌습니다.
대응	워크스페이스를 고객사 단위 테넌트로 두고, 워크스페이스·프로젝트 단위 다층 권한을 설계했습니다. 모든 권한 판단을 한 곳에서만 처리하도록 중앙화했습니다.
결과	**이후 권한 오류가 거의 사라졌고, 새 기능을 추가할 때 권한 규칙은 한 곳만 고치면 되도록 바뀌었습니다.**

02 구독 결제 · 쿠폰 · 다통화 설계

문제	카드 정보를 서버에 저장하지 않으면서 자동 갱신, 플랜·좌석 수 변경, 쿠폰, 다통화(KRW·USD·JPY), 실패 재청구를 모두 지원하는 구독 결제가 필요했습니다.
대응	PortOne 빌링키로 결제를 미리 예약하고 플랜·좌석·통화·쿠폰 정보를 DB에 먼저 저장한 뒤, 웹훅 결과와 매핑해 구독 상태를 갱신했습니다. 신규·자동갱신·플랜 변경·재청구를 하나의 결제 레이어로 통합하고, 업그레이드 차액은 잔여 기간 기준으로 계산했습니다. 통화는 CurrencyType으로 추상화했고, 쿠폰은 구독 유형별 할인율과 사용 횟수를 분리해 이메일 기준 중복 사용을 막았습니다. 결제는 DB 상태를 먼저 바꾼 뒤 PG를 호출하고, 실패하면 보상 트랜잭션으로 원복했습니다.
결과	**5가지 이상의 결제 시나리오를 실서비스에 적용했습니다. 운영 기간 결제 오류는 3건, 네트워크 장애를 빼면 1건이었으며 모두 정합성을 지킨 채 처리했습니다.**

03 고객 계약 · 라이선스 데이터 정합성 확보

문제	조회 성능을 위해 캐시를 도입한 뒤, 특수 계약 고객의 라이선스 정보가 반영되지 않는 오류가 실서비스에서만 발생했습니다. 캐시 유지 시간을 조정해도 해결되지 않았습니다.
대응	원인은 일반 계약만 가정한 무효화 구조였습니다. 라이선스·계약 데이터가 바뀌는 모든 지점에서 캐시를 즉시 무효화하도록 다시 설계했습니다.
결과	**같은 유형의 오류는 다시 발생하지 않았습니다.**

PROJECT 01 · REAT (계속)

04 어드민 · 운영 데이터 시스템 (풀스택)

문제	운영팀이 고객사 라이선스와 워크스페이스를 관리하고, 서비스 지표를 확인할 내부 도구가 필요했습니다.
대응	React 기반 어드민 웹과 API를 개발해 라이선스 관리, 워크스페이스 이전, 엔터프라이즈 고객 맞춤 기능을 넣었습니다. REAT·TOONDA의 사용자 활동 데이터를 자동 수집해 Looker Studio 대시보드로 시각화했습니다. 실제 사용 시간이 20% 수준인 어드민·개발 서버는 단순하고 인수인계가 쉬운 방식으로 운영 비용을 80% 이상 줄였습니다.
결과	**서비스 사용량과 운영 데이터를 시각화해 의사결정을 지원했습니다.**

05 실시간 고객 알림과 수신 설정

문제	1분 주기 Polling 알림은 주기를 줄이면 요청이 급증하고, 늘리면 알림이 늦어졌습니다. 업로드 진행률을 실시간으로 보고 싶다는 CS 요청도 늘었습니다.
대응	단방향 알림에 충분하고 운영 부담이 작은 SSE를 WebSocket 대신 채택했습니다. Redis에 사용자별 마지막 알림 ID를 저장해 재연결 시 놓친 알림을 다시 보냈고, 같은 구조를 업로드 진행률에도 재사용했습니다. 이메일 수신 설정 7종은 Boolean 컬럼 대신 비트마스크 정수 필드 1개로 통합했습니다.
결과	**반복 Polling 요청을 없애고, 연결이 끊겨도 유실 없이 실시간 알림을 제공했습니다. 수신 설정 컬럼은 7개에서 1개로 줄었습니다.**

06 리스트 조회 N+1 해소와 Redis 캐시

문제	아트워크 리스트 API 한 번에 연관 엔티티 조회로 최대 75개의 쿼리가 실행됐습니다.
대응	JOIN FETCH로 연관 엔티티를 한 쿼리로 묶고 Cache-Aside 패턴의 Redis 캐시를 더했습니다. Redis 장애에 대비해 헬스체크를 두고, 연속 실패 시 이메일 알림과 함께 DB로 자동 전환되게 했습니다. AOP와 Hibernate StatementInspector로 API별 쿼리 수와 응답 시간을 측정해 개선 지점을 찾았습니다.
결과	**쿼리 수가 75개에서 5개로 93% 줄었고, Redis 장애 중에도 서비스가 정상 동작했습니다.**

07 대용량 이미지 처리 Lambda 분리

문제	고해상도 이미지를 여러 장 올리면 처리에 5분 이상 걸렸고, 픽셀 전체를 메모리에 올리는 구조라 동시 업로드 시 OOM이 발생했습니다.
대응	요청이 불규칙하고 오래 걸리는 작업이라 독립 확장이 되는 Lambda로 분리했습니다. 여러 장은 CompletableFuture로 병렬 처리하고 S3 업로드에 논블로킹 I/O를 적용했으며, 원본은 S3 Lifecycle로 자동 삭제했습니다.
결과	**20장 기준 처리 시간이 5분에서 15초로 줄었고 2GB 이상 업로드 시의 OOM이 사라졌습니다. 이후 조각 이미지 주소를 규칙 기반 재생성으로 바꿔 DB 조회 속도를 70% 개선했습니다.**

PROJECT 01 · REAT (계속)

08 Lambda 분산 구조에서 Spring Boot 중앙 서버로 전환

문제	초기 서비스는 Lambda 함수 약 74개로 나뉘어 있어 함수 간 호출 중 동시 수정·조회에서 정합성 문제가 생겼고, Node.js와 Spring 코드가 섞여 일관성이 떨어졌습니다. 콜드 스타트 지연 CS도 이어졌습니다.
대응	도메인 단위 레이어드 아키텍처의 Spring Boot 중앙 서버를 제안하고, 인증·인가를 JWT로 통합했습니다. 출시 일정 때문에 반대 의견이 있었지만, 3일 만에 서버 구축과 로그인까지 PoC로 보여 전환 결정을 이끌었습니다.
결과	**콜드 스타트가 사라져 초기 응답이 빨라졌고, 정합성 문제를 구조적으로 해결했습니다. 트래픽에 따른 비용도 예측 가능해졌습니다.**

09 AWS에서 GCP로 인프라 이전

문제	AWS 크레딧이 소진되면서 월 약 150만원의 인프라 비용이 발생하게 됐습니다.
대응	GCP 스타트업 크레딧으로 비용을 대체할 수 있음을 확인하고, DB → 백엔드 → 프론트엔드 순으로 단계 이전했습니다. DB는 pg_dump/pg_restore로 미리 적재한 뒤 전환 시점에 기존 RDS를 Read-Only로 바꿔 증분을 맞췄습니다. S3에 평문으로 있던 자격증명은 Secret Manager로 옮기고 서비스 계정 최소 권한을 적용했으며, 트래픽은 DNS A 레코드 변경으로 전환했습니다.
결과	**각 전환을 주말 야간 4시간 이내에 데이터 손실 없이 마쳤고, 월 인프라 비용을 약 150만원에서 3만원으로 줄였습니다(98% 절감).**

10 Jenkins CI/CD 파이프라인

문제	배포가 수작업이라 잘못된 이미지 배포, Auto Scaling 축소 누락, 환경 변수 누락이 간헐적으로 생겼습니다.
대응	GitHub Actions와 Jenkins를 연동해 main 푸시 시 테스트 후 파이프라인이 돌게 하고, 환경 변수는 외부 주입으로 분리했습니다. 개발 서버는 롤링 업데이트로 자동 배포하고, 운영 서버는 배포 시점을 통제하려고 수동 실행으로 남겼습니다.
결과	**반복되던 배포 실수가 사라졌고, 개발 생산성과 운영 안정성을 함께 확보했습니다.**

사이드 · 교육 프로젝트

PROJECT 02 · 사이드 프로젝트

나공주 키우기 — 캐릭터 육성형 집중 앱

2026.09 – 진행 중

역할 · 기술	백엔드 설계 (2인) · Expo(React Native), TypeScript, PostgreSQL · 2027.03 출시 예정
주요 내용	• 기기가 먼저 정산해 보상을 보여 주고, 서버가 같은 도메인 엔진으로 재정산해 확정·병합하는 오프라인 퍼스트 동기화를 설계했습니다. • session:{id}, plan:{id}처럼 기기와 서버가 같은 멱등키를 만들고, DB 유일 제약으로 재전송·동시 달성에도 한 번만 지급되게 했습니다. • 결과가 갈리는 위험 6곳(난수 배수, 벽시계 의존 등)을 진단하고, 확정값이 다를 때 원장 조정 줄과 알림을 남기는 정책을 제안했습니다. 설계 문서 4건 작성, 서버 구현은 다음 단계입니다.

사이드 · 교육 프로젝트 (계속)

PROJECT 03 · KDT 팀 프로젝트
고글에듀 — 자격증·멘토링 예약 플랫폼 2026.04 - 2026.05

역할 · 기술	6인 팀 리더 · 최종 프로젝트 우수상 · Spring Boot, PostgreSQL, Redis, Kafka, GCP, GitHub Actions, K6
주요 내용	• 비관적·낙관적 락의 한계를 비교해 Redisson 분산 락(Watchdog)으로 멀티 인스턴스의 인원 초과·중복 신청을 막았습니다. • pg_trgm GIN 인덱스로 117,000건 기준 평균 응답 726ms → 163ms(77% 감소), p99 1,654ms → 408ms로 개선했습니다. • Redis 3대·Sentinel 3대로 자동 Failover를 구성해 락·캐시의 단일 장애 지점을 없앴습니다.

PROJECT 04 · KDT 팀 프로젝트
팔로우미 배송 — MSA 물류 배송 서비스 2026.03 - 2026.04 (2주)

역할 · 기술	6인 팀 리더 · Spring Boot, PostgreSQL, Kafka, jOOQ, Eureka, Docker
주요 내용	• Outbox에 저장 후 커밋 뒤 발행, 실패 시 재발행·보상 트랜잭션, 소비자는 Inbox 기본키로 멱등 처리하는 구조를 공통 라이브러리로 만들어 전 서비스에 적용했습니다. • 쓰기는 JPA, 조회는 jOOQ로 나눠 상태 기반 복합 필터링을 타입 안전하게 구현하고, 2-query 페이지네이션으로 N+1을 피했습니다.

PROJECT 05 · 사이드 프로젝트
Trobl — 부부싸움 녹음·분석 앱 2025.05 - 2025.11 · App Store 출시

역할 · 기술	백엔드 · 어드민 웹 · Spring Boot, PostgreSQL, Redis, Firebase, GCP, React
주요 내용	• React 어드민 웹으로 사용자·사용량 대시보드와 앱 내 콘텐츠 게시 기능을 만들었습니다. • 즉시·배치·예약 알림을 Redis 큐와 스케줄러로 나누고, 몰리는 알림은 10분마다 사용자별 1건으로 묶어 보내는 FCM 파이프라인을 구축했습니다. • Google·Kakao·Apple OAuth를 Provider별 특성을 살린 단일 로그인 구조로 통합했습니다.

위에 기재한 내용은 사실과 다름없음을 확인합니다. 2026년 10월 24일 지원자 **하 지 혜**